

Software Engineering Project Report

Film Photography Submission
Management System

CNPM PROJECT TEAM¹

17th September 2026

¹<https://github.com/vyn141367-collab/CNPM-SUM-2026>

Dedicated to our project supervisor and team members for their continuous support throughout the software development lifecycle.

Contents

1	Project Introduction	2
1.1	Background & Context	2
1.2	Project Objectives	2
1.3	Project Scope	2
2	Project Management Plan	4
2.1	Team Organization & Roles	4
2.2	Development Methodology	4
2.3	Risk Management	5
3	Software Requirement Specification	6
3.1	Actors & Stakeholders	6
3.2	Functional Requirements (FR)	6
3.3	Non-Functional Requirements (NFR)	7
4	System Design	8
4.1	System Architecture	8
4.2	Database Design (Entity-Relationship Diagram)	8
4.3	Sequence Diagram: Submission Workflow	9
4.4	Database Design (Entity-Relationship Diagram)	9
5	API Specification	11
5.1	Endpoint Summary	11
5.2	Sample Payloads	11
5.2.1	POST /api/submissions Request	11
5.2.2	Response (201 Created)	12
6	UI/UX Design & User Manual	13
6.1	UI Design Principles	13
6.2	User Manual	13
6.2.1	Submitting an Entry (Contestant)	13
6.2.2	Grading Entries (Judge)	14

7	Software Testing	15
7.1	Test Strategy	15
7.2	Test Cases	15
8	Deployment & Conclusion	16
8.1	System Deployment	16
8.1.1	Prerequisites	16
8.1.2	Execution Steps	16
8.2	Conclusion & Future Work	16

List of Figures

4.1	System Architecture Diagram	8
4.2	Entity-Relationship Diagram (ERD)	9
4.3	Sequence Diagram for Image Submission	9
4.4	Entity-Relationship Diagram (Database Relational Schema) .	10

List of Tables

2.1	Risk Assessment and Mitigation Plan	5
5.1	API Endpoints Overview	11
7.1	System Test Cases	15

Preface

This document presents the complete software engineering report for the Film Photography Submission Management System, built using Streamlit, Flask, and SQLite.

Structure of the Document

- **Chapters 1–3:** Project Introduction, Management Plan, and Software Requirement Specification.
- **Chapters 4–5:** System Design (with TikZ Architecture, ERD, and Sequence Diagrams) and API Specification.
- **Chapter 6:** UI/UX Design and User Manual.
- **Chapters 7–8:** Testing Strategy, Deployment Guide, and Conclusion.

Acknowledgements

We express our appreciation to our instructor and peer reviewers for their guidance.

Chapter 1

Project Introduction

1.1 Background & Context

Film photography has experienced a significant revival among photography enthusiasts in recent years. However, managing film photography contests traditionally involves fragmented manual processes, including collecting large image files, validating photographic parameters (camera, film stock, format), and distributing submissions to judges.

To solve this issue, the **Film Photography Submission Management System** is developed to digitalize and streamline the entire process from submission to evaluation and administrative control.

1.2 Project Objectives

The primary goals of the project include:

- Providing a simple, responsive portal for contestants to upload their artworks alongside metadata (film stock, camera, format).
- Implementing client-side image optimization to reduce transmission latency.
- Creating an intuitive interface for judges to evaluate submissions efficiently.
- Offering an administrative dashboard for system-wide monitoring and data management.

1.3 Project Scope

The scope of this software engineering project includes:

- **In Scope:** User registration and submission form, client-side image compression, automated mock/API validation, judge scoring portal, administrative dashboard, RESTful API backend, and SQLite database storage.
- **Out of Scope:** Physical film processing/scanning services, advanced payment gateway integration, and real-time live streaming.

Chapter 2

Project Management Plan

2.1 Team Organization & Roles

The development team consists of members assuming the following key roles:

- **Project Manager / Scrum Master:** Coordinates tasks, manages project timeline, and oversees documentation.
- **Frontend Developer:** Builds web user interfaces using Streamlit for User, Judge, and Admin portals.
- **Backend Developer:** Designs and implements RESTful APIs using Flask and manages SQLite database integrations.
- **QA / Tester:** Conducts functional testing, integration testing, and verifies API behavior.

2.2 Development Methodology

The project follows an **Agile / Scrum** development model divided into multiple short sprints:

- **Sprint 1:** Requirements gathering, system architecture design, and database schema setup.
- **Sprint 2:** Backend API development and initial Streamlit user portal implementation.
- **Sprint 3:** Integration of client-side image compression, AI validation module, and judge scoring portal.
- **Sprint 4:** Testing, bug fixing, performance optimization, and final documentation.

2.3 Risk Management

Risk	Impact	Mitigation Strategy
Large image upload timeouts	High latency or UI freeze	Implement client-side image compression using PIL before HTTP request.
Backend server failure	System unavailable	Run health checks and provide informative error handling on Frontend.
API latency from external AI	Delayed submission response	Add loading indicators and fallback mock validation.

Table 2.1: Risk Assessment and Mitigation Plan

Chapter 3

Software Requirement Specification

3.1 Actors & Stakeholders

The system defines three major primary actors:

- **Contestant (User):** Submits film photography artworks along with technical metadata.
- **Judge:** Reviews assigned submissions, views detailed image information, and submits scores.
- **Administrator (Admin):** Oversees the entire system, manages entries, and monitors contest metrics.

3.2 Functional Requirements (FR)

- **FR-01 (Submission):** The system shall allow users to upload images (JPG, PNG, WEBP) along with metadata (Name, Title, Film Stock, Camera, Format).
- **FR-02 (Image Compression):** The system shall compress image files on the client side prior to payload transmission.
- **FR-03 (Validation):** The system shall validate submitted entries via automated verification rules or external API checks.
- **FR-04 (Evaluation):** The system shall allow judges to review images and submit ratings (1–10 scale) with comments.
- **FR-05 (Management):** The system shall provide admins with tools to list, filter, and delete contest entries.

3.3 Non-Functional Requirements (NFR)

- **NFR-01 (Performance):** Image submission requests should complete within 3 seconds after client-side compression.
- **NFR-02 (Usability):** The interface must be intuitive, dark-mode friendly for photography viewing, and accessible without extensive training.
- **NFR-03 (Reliability):** The backend RESTful API must return appropriate standard HTTP status codes (200, 201, 400, 500).

Chapter 4

System Design

4.1 System Architecture

The system adopts a decoupled Client-Server architecture. The Presentation Layer is built using Streamlit, communicating with a Flask RESTful API Backend via HTTP protocols. Data is stored in an SQLite relational database alongside a localized media store.

Streamlit Frontend Layer

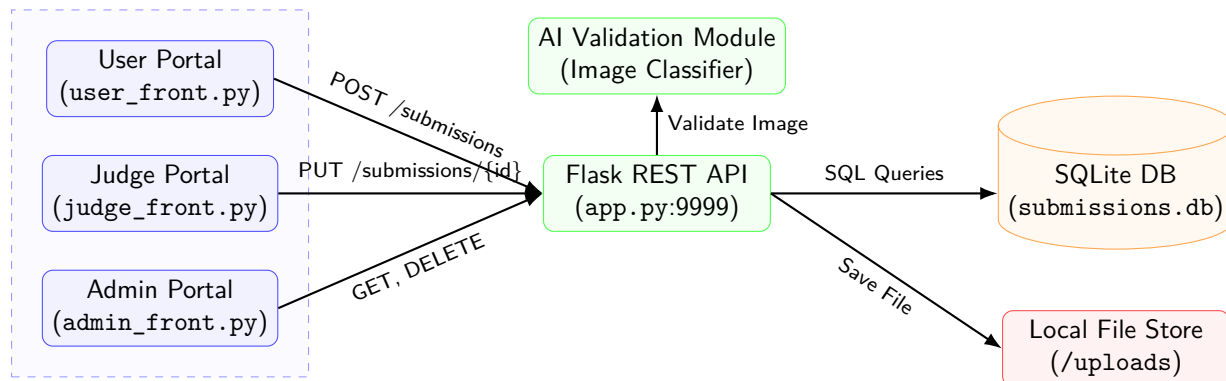


Figure 4.1: System Architecture Diagram

4.2 Database Design (Entity-Relationship Diagram)

The relational schema comprises three main entities: `USERS`, `SUBMISSIONS`, and `EVALUATIONS`.

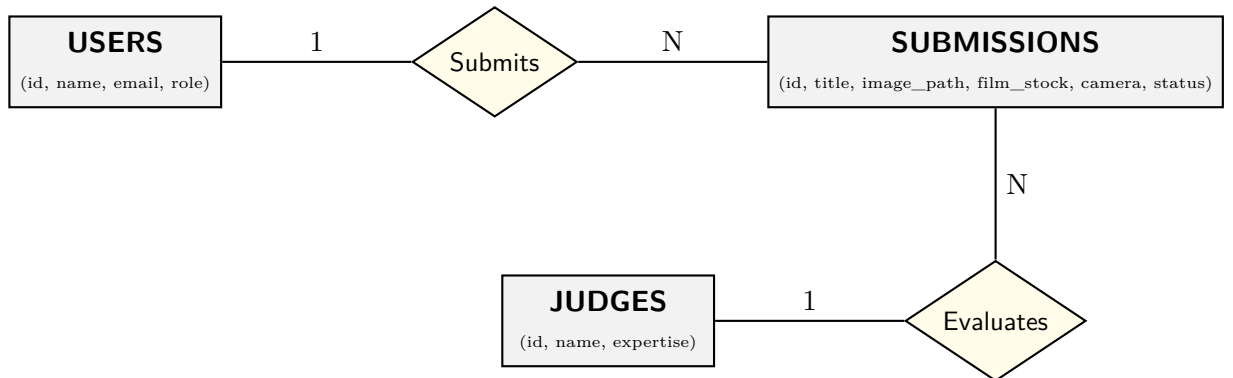


Figure 4.2: Entity-Relationship Diagram (ERD)

4.3 Sequence Diagram: Submission Workflow

The sequence below illustrates client-side image compression followed by asynchronous REST interaction with the Flask Backend.

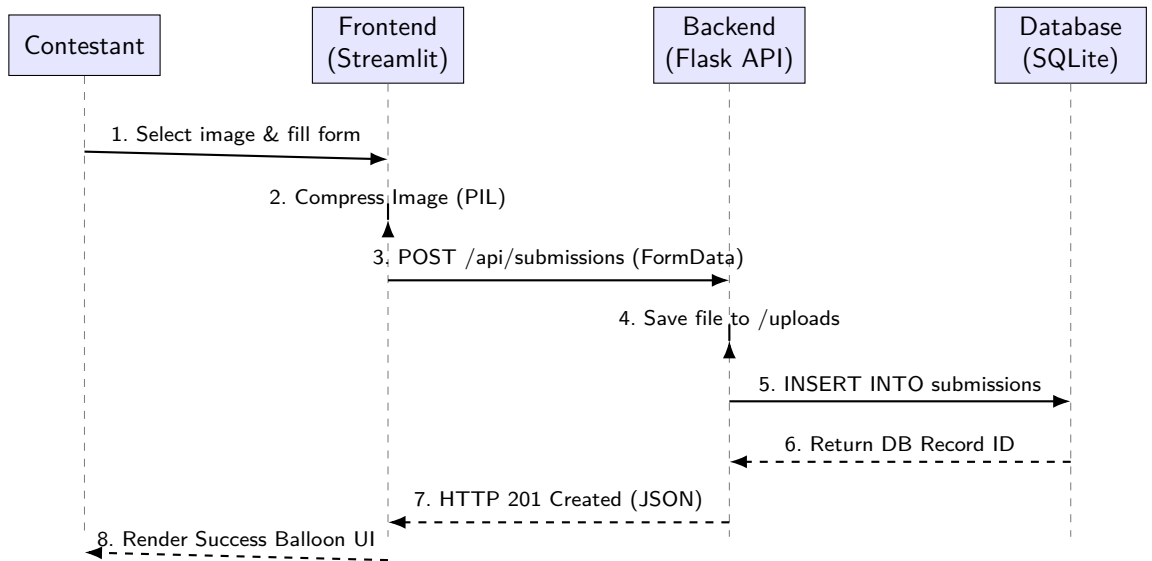


Figure 4.3: Sequence Diagram for Image Submission

4.4 Database Design (Entity-Relationship Diagram)

The SQLite database consists of three main entities: **USERS**, **SUBMISSIONS**, and **EVALUATIONS**. The diagram below illustrates the relational table structures, primary keys (PK), foreign keys (FK), and their cardinality relationships.

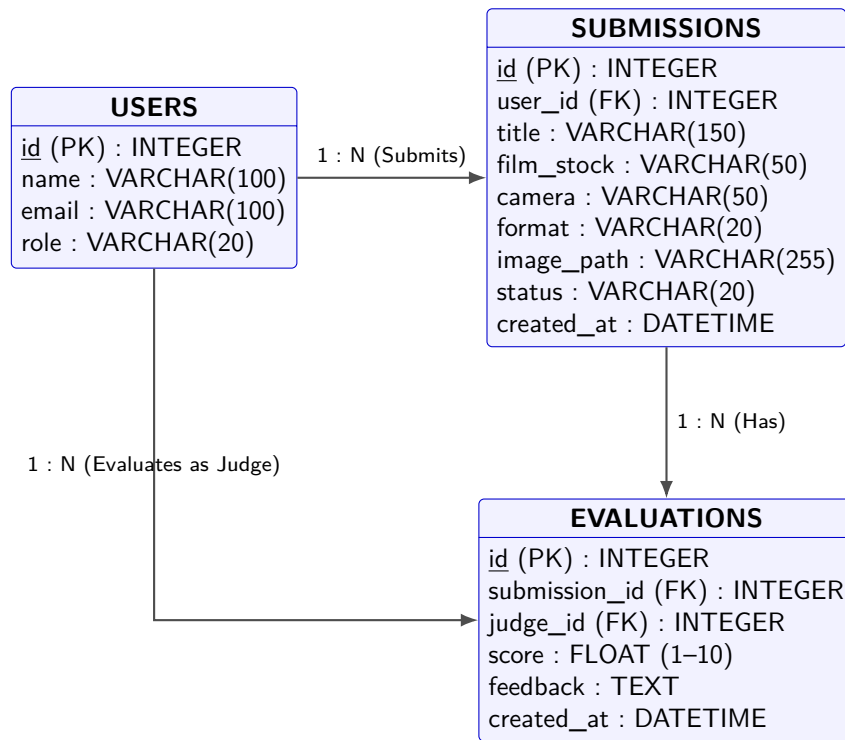


Figure 4.4: Entity-Relationship Diagram (Database Relational Schema)

Chapter 5

API Specification

5.1 Endpoint Summary

The Flask API communicates over standard HTTP protocols, accepting form-data and returning JSON payloads.

HTTP Method	Endpoint Path	Functional Description
POST	/api/submissions	Accepts submission metadata and image file.
GET	/api/submissions	Retrieves all submissions with scores.
PUT	/api/submissions/{id}	Updates score and evaluation notes.
DELETE	/api/submissions/{id}	Deletes an entry from DB and filesystem.

Table 5.1: API Endpoints Overview

5.2 Sample Payloads

5.2.1 POST /api/submissions Request

Multipart form-data fields:

- name: "Jane Doe"
- title: "Sunset on Kodak Gold"
- film_stock: "Kodak Gold 200"
- camera: "Leica M6"
- file: Binary image data (JPEG)

5.2.2 Response (201 Created)

```
{
  "status": "success",
  "message": "Submission created successfully",
  "data": {
    "id": 102,
    "title": "Sunset on Kodak Gold",
    "image_path": "/uploads/102_sunset.jpg"
  }
}
```

Chapter 6

UI/UX Design & User Manual

6.1 UI Design Principles

The interfaces are built using Streamlit with a dark theme optimized for viewing photography:

- **User Portal:** Single-column form structure with clear upload status and warning indicators.
- **Judge Portal:** Grid-based layout displaying high-resolution photos alongside metadata cards and score sliders.
- **Admin Portal:** Dashboard layout with summary metrics and editable data tables.

6.2 User Manual

6.2.1 Submitting an Entry (Contestant)

1. Open the User Portal in a browser.
2. Fill in contestant information (Name, Artwork Title, Film Stock, Camera).
3. Click **Browse files** to select a film photo (PNG, JPG, WEBP).
4. Click **Submit**. Client-side compression executes automatically before sending data to the server.

6.2.2 Grading Entries (Judge)

1. Open the Judge Portal.
2. Select an ungraded photo from the list.
3. Review image details and EXIF metadata.
4. Adjust the score slider (1–10), enter comments, and click **Save Evaluation**.

Chapter 7

Software Testing

7.1 Test Strategy

Testing focuses on verifying image compression efficiency, API response correctness, and database consistency.

7.2 Test Cases

Test ID	Action / Input	Expected Outcome	Status
TC-01	Submit form with empty "Name" field.	UI displays missing field warning; API call blocked.	PASS
TC-02	Upload 3.7MB photo file.	Client compresses image to <300KB and POST succeeds (201).	PASS
TC-03	Judge submits valid score (8/10).	DB updates score field; entry disappears from ungraded queue.	PASS
TC-04	Admin triggers DELETE request.	Database record and local image file are both removed.	PASS

Table 7.1: System Test Cases

Chapter 8

Deployment & Conclusion

8.1 System Deployment

8.1.1 Prerequisites

Python 3.9+ environment with required packages installed via:

```
pip install -r requirements.txt
```

8.1.2 Execution Steps

1. Start Backend API: `python src/app.py` (Port 9999).
2. Launch Admin Portal: `streamlit run admin_front.py -server.port 8501` (Port 8501).
3. Launch Judge Portal: `streamlit run judge_front.py -server.port 8502` (Port 8502).
4. Launch User Portal: `streamlit run user_front.py -server.port 8503` (Port 8503).

8.2 Conclusion & Future Work

- **Conclusion:** The system successfully digitalizes film photography submission and judging workflows. Client-side compression effectively eliminates network timeouts caused by large photo files.
- **Future Work:** Implement JWT user authentication, deploy cloud storage for uploaded assets (e.g., AWS S3), and train a custom vision model for automated film border detection.